System Programming Project 4

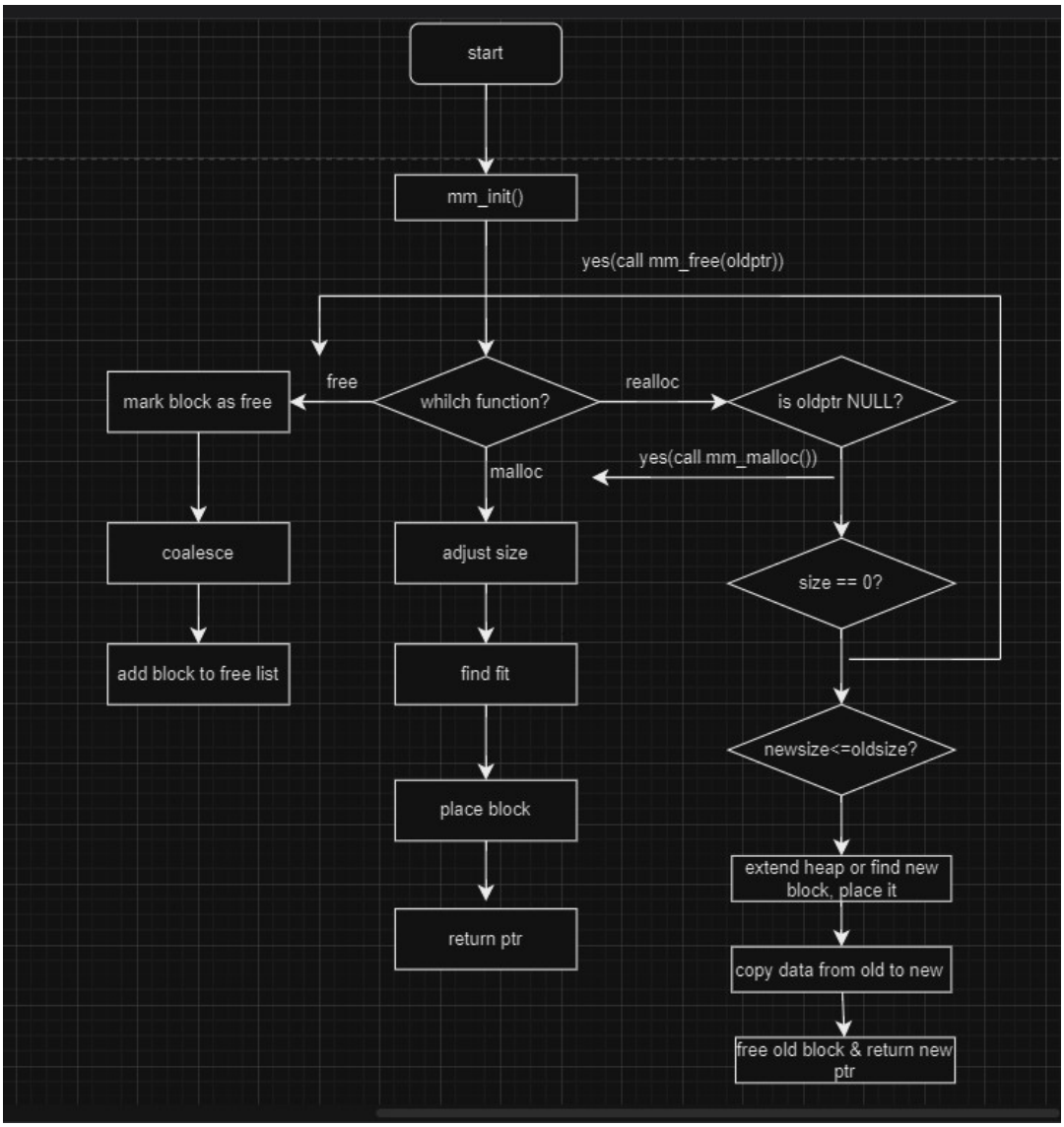
담당 교수 : 김영재 교수님
이름 : 김민우
학번 : 20212019

1. 개발 목표

동적 메모리 할당기 (mm.c)의 개발 목표는 다음과 같다. 우선 정확성을 보장하여 mm_malloc, mm_free, mm_realloc 함수가 명세에 맞게 오류 없이 작동하도록 한다. 둘째, 효율성을 높여 메모리 공간 활용도를 극대화하고 할당 및 해제 요청을 신속하게 처리한다. 셋째, 재사용성을 확보하여 coalesce 함수를 통해 인접한 자유 블록을 병합하고, 자유 블록 리스트를 효율적으로 관리하여 메모리 블록을 재사용할 수 있게 한다. 넷째, 유연성을 강화하여 다양한 크기의 메모리 요청을 처리하고, mm_realloc 함수가 기존 블록 크기를 조정하거나 새로운 블록을 할당할 수 있도록 한다. 다섯째, 디버깅 및 유지보수를 용이하게 하기 위해 힙 일관성 검사기를 구현하고, 코드 가독성을 높이며 모듈화를 통해 유지보수를 쉽게 한다. 마지막으로, 최적화를 통해 find_fit 함수와 메모리 관리 알고리즘을 개선하여 성능을 향상시킨다. 이를 통해 성능이 뛰어나고 신뢰할 수 있는 동적 메모리 할당기를 구현하는 것이 목표이다.

2. 개발 범위 및 내용

(1) flow chart



이 플로우차트는 동적 메모리 할당기(mm.c)의 주요 함수인 mm_malloc, mm_free, mm_realloc의 작동 방식을 시각적으로 표현한 것이다. 프로그램이 시작되면 먼저 mm_init() 함수가 호출되어 힙을 초기화하고 heap_listp를 설정한다. 초기 힙을 확장하여 첫 번째 블록을 설정한 후, 호출된 함수에 따라 free, malloc, realloc 중 하나로 분기한다.

mm_free(bp) 함수에서는 전달된 포인터(bp)가 가리키는 블록을 해제한다. 블록의 header와 footer를 업데이트하여 해당 블록이 자유 블록임을 표시하고, 인접한 자유 블록과 병합하여 더 큰 자유 블록을 만든다. 이를 통해 단편화를 줄이고 메모리 활용 효율을 높인 후, 병합된 블록을 자유 리스트에 추가한다.

mm_malloc(size) 함수에서는 요청된 메모리 크기(size)를 조정하여 8바이트 배수로 맞춘다. 이는 메모리 정렬을 위해 필요하다. 그런 다음 자유 리스트에서 적절한 블록을 찾는다. 만약 적절한 블록이 발견되면 해당 블록을 배치하고 자유 리스트에서 제거한다. 발견되지 않으면 힙을 확장한 후, 새로운 블록을 배치한다. 마지막으로 할당된 블록의 포인터를 반환한다.

mm_realloc(oldptr, size) 함수는 먼저 oldptr가 NULL인지 확인한다. NULL인 경우, mm_malloc(size)를 호출하여 새로운 블록을 할당하고 포인터를 반환한다. 만약 size가 0이면 mm_free(oldptr)을 호출하여 기존 블록을 해제하고 NULL을 반환한다. 새로 요청된 크기(newsize)가 기존 블록 크기(oldsize)보다 작거나 같으면 기존 포인터를 반환한다. 그렇지 않으면 힙을 확장하거나 새로운 블록을 찾아 배치한 후, 기존 블록의 데이터를 새로운 블록으로 복사합니다. 마지막으로 기존 블록을 해제하고 새로운 블록의 포인터를 반환한다.

(2) 주요 함수 설명

* mm_init :

mm_init 함수는 메모리 할당기의 초기화를 담당한다. 이 함수는 힙 영역을 초기화하고, 프롤로그와 에필로그 블록을 설정한다. 먼저 mem_sbrk 함수를 호출하여 8워드(32바이트)의 메모리를 요청하고, 이 공간을 이용해 힙의 시작 부분에 프롤로그 헤더와 풋터, 에필로그 헤더를 배치한다. 그런 다음, extend_heap 함수를 호출하여 초기 힙 공간을 확장하고, 필요한 경우 더 많은 메모리를 확보한다. 초기화가 성공적으로 완료되면 0을 반환하고, 그렇지 않으면 -1을 반환한다.

* mm_malloc :

mm_malloc 함수는 요청된 크기의 메모리 블록을 할당한다. 먼저, 요청된 크기 size를 고려하여 실제 필요한 블록 크기 asize를 계산한다. 이는 최소 블록 크기와 8바이트 정렬을 고려한 크기다. 그 다음, 자유 리스트에서 적절한 블록을 찾기 위해 find_fit 함수를 호출한다. 만약 적절한 블록을 찾으면, place 함수를 호출하여 해당 블록을 배치하고, 포인터를 반환한다. 적절한 블록을 찾지 못한 경우, 힙을 확장하기 위해 extend_heap 함수를 호출하고, 새로운 블록을 배치한 후 포인터를 반환한다.

* mm_free :

mm_free 함수는 할당된 메모리 블록을 해제한다. 전달된 포인터 bp가 NULL이 아닌 경우, 해당 블록의 헤더와 풋터를 업데이트하여 블록을 자유 블록으로 표시한다. 그런 다음, coalesce 함수를 호출하여 인접한 자유 블록과 병합함으로써 단편화를 줄이고 메모리 관리 효율성을 높인다.

* mm_realloc :

mm_realloc 함수는 기존 메모리 블록의 크기를 변경한다. 만약 oldptr가 NULL이면, 이는 mm_malloc(size) 호출과 동일하게 처리된다. 만약 size가 0이면, 이는 mm_free(oldptr) 호출과 동일하게 처리되며, NULL을 반환한다. 기존 블록의 크기보다 새로운 크기가 작거나 같으면, 기존 블록을 그대로 사용하고 포인터를 반환한다. 그렇지 않은 경우, 다음 블록이 자유 상태이고 병합하여 충분한 크기를 확보할 수 있으면, 병합하여 기존 블록을 확장한다. 이 경우도 기존 포인터를 반환한다. 이러한 조건이 충족되지 않으면, 새로운 블록을 할당하고 기존 데이터의 일부를 새로운 블록으로 복사한 후, 기존 블록을 해제하고 새로운 포인터를 반환한다.

* coalesce :

coalesce 함수는 주어진 블록을 주변의 자유 블록과 병합한다. 먼저, 이전 블록과 다음 블록의 할당 상태를 확인한다. 이전 블록과 다음 블록이 모두 할당된 경우, 현재 블록을 자유 리스트에 추가하고 반환한다. 이전 블록은 할당되고 다음 블록은 자유 상태인 경우, 다음 블록과 병합하여 하나의 큰 자유 블록으로 만든다. 이전 블록은 자유 상태이고 다음 블록이 할당된 경우, 이전 블록과 병합한다. 이전 블록과 다음 블록이 모두 자유 상태인 경우, 현재 블록을 이전 블록과 다음 블록과 모두 병합하여 더 큰 자유 블록으로 만든다. 병합된 블록을 자유 리스트에 추가한 후 반환한다.

* extend_heap :

extend_heap 함수는 힙을 확장한다. 요청된 워드 수를 바이트 크기로 변환하여 메모리를 요청하고, 새로운 블록의 헤더와 풋터를 설정한다. 에필로그 블록을 갱신하고, 새로 할당된 블록을 병합한다. 확장이 성공적으로 완료되면 새로 할당된 블록의 포인터를 반환하고, 실패하면

NULL을 반환한다.

3. 구현 결과 및 성능 평가 결과

(1) 구현 결과

```c
static void remove_free(void *bp)
{
    if (PREV_FP(bp) != NULL)
    {
        NEXT_FP(PREV_FP(bp)) = NEXT_FP(bp);
    }
    else
    {
        heap_listp = NEXT_FP(bp);
    }
    if (NEXT_FP(bp) != NULL)
    {
        PREV_FP(NEXT_FP(bp)) = PREV_FP(bp);
    }
    PREV_FP(bp) = NULL;
    NEXT_FP(bp) = NULL;
}
```

```c
static void add_free(void *bp)
{
    NEXT_FP(bp) = heap_listp;
    PREV_FP(bp) = NULL;
    if (heap_listp != NULL)
    {
        PREV_FP(heap_listp) = bp;
    }
    heap_listp = bp;
}
```

```c
void *mm_realloc(void *oldptr, size_t size)
{
    size_t oldsize, newsize;
    void *newptr;

    if ((int)size < 0)
    {
        return NULL;
    }
    if (size == 0)
    {
        mm_free(oldptr);
        return NULL;
    }
    if (oldptr == NULL)
    {
        return mm_malloc(size);
    }

    oldsize = GET_SIZE(HDRP(oldptr));
    newsize = size + 2 * WSIZE;

    if (newsize <= oldsize)
    {
        return oldptr;
    }
```

```c
    size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKP(oldptr)));
    size_t next_size = GET_SIZE(HDRP(NEXT_BLKP(oldptr)));
    if (!next_alloc && (oldsize + next_size) >= newsize)
    {
        remove_free(NEXT_BLKP(oldptr));
        PUT(HDRP(oldptr), PACK(oldsize + next_size, 1));
        PUT(FTRP(oldptr), PACK(oldsize + next_size, 1));
        return oldptr;
    }

    newptr = mm_malloc(size);
    if (newptr == NULL)
    {
        return NULL;
    }
    memcpy(newptr, oldptr, oldsize - WSIZE);
    mm_free(oldptr);
    return newptr;
}
```

이번 구현에서는 explicit free list를 사용하여 메모리 할당과 해제를 더욱 효율적으로 관리하기 위해 새로운 함수들인 add_free, remove_free, 그리고 mm_realloc을 추가했다. add_free 함수는 주어진 블록 포인터 bp를 free list에 추가하는 역할을 한다. 이 함수는 먼저 bp의 다음 블록 포인터를 현재의 free list 첫 번째 블록으로 설정하고, bp의 이전 블록 포인터를 NULL로 설정한다. 만약 현재 free list가 비어 있지 않다면, free list의 첫 번째 블록의 이전 포인터를 bp로 설정한다. 마지막으로, heap_listp를 bp로 갱신하여 bp가 새로운 free list의 첫 번째 블록이 되도록 한다.

remove_free 함수는 주어진 블록 포인터 bp를 free list에서 제거하는 역할을 한다. 이 함수는 먼저 bp가 free list의 첫 번째 블록인지 확인한다. 만약 그렇다면, heap_listp를 bp의 다음 블록으로 설정하고, 이전 포인터를 NULL로 설정한다. 그렇지 않은 경우, bp의 이전 블록의 다음 블록 포인터를 bp의 다음 블록으로 설정하고, bp의 다음 블록의 이전 블록 포인터를 bp의 이전 블록으로 설정하여 bp를 free list에서 제거한다. 이 과정을 통해 free list에서 bp를 효율적으로 제거할 수 있다.

마지막으로, mm_realloc 함수는 기존 메모리 블록의 크기를 변경하는 역할을 한다. 이 함수는 먼저 size가 0이거나 음수인 경우, mm_free(oldptr)를 호출하거나 NULL을 반환한다. oldptr가 NULL인 경우, mm_malloc(size)를 호출하여 새로운 메모리 블록을 할당한다. 기존 블록의 크기보다 새로운 크기가 작거나 같으면, 기존 블록을 그대로 사용하고 포인터를 반환한다. 만약 다음 블록이 자유 상태이고 병합하여 충분한 크기를 확보할 수 있으면, 병합하여 기존 블록을 확장한다. 이러한 조건이 충족되지 않으면, 새로운 블록을 할당하고 기존 데이터

를 새로운 블록으로 복사한 후, 기존 블록을 해제하고 새로운 포인터를 반환한다. 이러한 방식으로 mm_realloc 함수는 메모리 재할당을 효율적으로 처리하게 된다.

(2) 성능 평가

```
Results for mm malloc:
trace  valid  util     ops      secs  Kops
 0       yes   89%    5694  0.000396 14368
 1       yes   92%    5848  0.000288 20277
 2       yes   94%    6648  0.000543 12254
 3       yes   96%    5380  0.000465 11575
 4       yes   66%   14400  0.000339 42440
 5       yes   88%    4800  0.000857  5601
 6       yes   85%    4800  0.002625  1829
 7       yes   55%   12000  0.013785   871
 8       yes   51%   24000  0.011746  2043
 9       yes   42%   14401  0.000738 19506
10       yes   53%   14401  0.000261 55197
Total          74%  112372  0.032044  3507

Perf index = 44 (util) + 40 (thru) = 84/100
```

이번 구현에서 일부 테스트 케이스에서 메모리 활용도가 50% 이하로 떨어지거나, 처리 시간이 길어지는 경우가 있었지만, 전체적인 성능 지표는 84/100으로 높은 점수를 받았다. 이는 explicit free list가 메모리 할당기에서 더 효율적이고 안정적인 성능을 제공한다는 것을 보여준다. 결론적으로, implicit list에 비해 explicit free list를 사용한 이번 구현은 메모리 활용도와 처리량 면에서 더 나은 성능을 발휘하였으며, 메모리 단편화 문제를 효과적으로 해결하였다.